Software Requirements Specification

for

Community Skill Exchange and Mentorship Platform

A Skill Matching and AI-Assisted Mentorship Platform

"Learn. Teach. Connect. Grow."

Version 1.0

Prepared by

Nehal Shah - Reg No. 2427030423

Tanishq Tiwari - Reg No. 2427030776

Course: Software Engineering Lab

Table of Contents

1. Introduction 1
1.1 Purpose 1
1.2 Document Conventions 1
1.3 Intended Audience and Reading Suggestions 2
1.4 Product Scope 2
1.5 References 2
2. Overall Description 3
2.1 Product Perspective 3
2.2 Product Functions 3
2.3 User Classes and Characteristics 3
2.4 Operating Environment 4
2.5 Design and Implementation Constraints 4
2.6 User Documentation 4
2.7 Assumptions and Dependencies 4
3. External Interface Requirements 5
3.1 User Interfaces 5
3.2 Hardware Interfaces 5
3.3 Software Interfaces 5
3.4 Communications Interfaces 5
4. System Features 6
4.1 Account and Profile Management 6
4.2 Availability and Skill Management 6
4.3 Search and Profile Matching 7
4.4 Mentorship Request and Session Management 8
4.5 Communication and Notifications 9
4.6 AI-Assisted Learning Plan and Session Summary 9
4.7 Feedback, Reporting and Administration 10
4.8 Dashboard and Activity Tracking 10
5. Other Nonfunctional Requirements 11
5.1 Performance Requirements 11
5.2 Safety Requirements 11
5.3 Security Requirements 11
5.4 Software Quality Attributes 11
5.5 Business Rules 12
6. Other Requirements 13
Appendix A: Glossary 14
Appendix B: Analysis Models 15
B.1 Use Case Model 15
B.2 Class Model 16
B.3 Object Model 17
B.4 Package Model 18
B.5 Component Model 19
B.6 Deployment Model 20

1.Introduction

Finding the right person to learn from is often as difficult as learning the skill itself. Students and working professionals may already have useful knowledge they can share, but opportunities for structured skill exchange are usually spread across informal groups, social networks, and disconnected platforms. A person looking for a mentor may therefore spend considerable time identifying someone with the right skill, compatible availability, and a suitable learning format.

The Community Skill Exchange and Mentorship Platform is proposed as a web-based system that brings these activities into one place. A user can maintain a profile, specify skills that they are able to teach, identify skills they want to learn, and state their preferred schedule and mode of interaction. The platform uses this information to identify suitable mentor-learner pairs and supports the interaction from an initial request through scheduled sessions and feedback.

The platform also includes selected AI-assisted functions. Profile matching can be supported by a compatibility model, learning goals can be converted into a structured learning plan, and completed sessions can be summarized into concise notes and follow-up actions. These functions are intended to reduce the amount of manual effort required from users while keeping the final decision with the participants.

1.1 Purpose

The purpose of this Software Requirements Specification is to define the functional and nonfunctional requirements of the Community Skill Exchange and Mentorship Platform. It establishes a common understanding of the proposed system for the project team, faculty evaluator, developers, testers, and future maintainers.

The system is intended to:

- provide a structured user profile for skills, goals, availability, and learning preferences;
- help users discover people who can teach a required skill or who need a skill the user can offer;
- rank and explain potential matches using information available in the two profiles;
- support mentorship requests, responses, scheduling, messaging, and session status;
- provide AI-assisted learning plans and session summaries;
- collect feedback and provide basic reporting and moderation facilities;
- maintain a consistent record of users, skills, requests, sessions, and related activity.

1.2 Document Conventions

The following terms and conventions are used throughout this SRS. Requirement statements use “shall” for mandatory behavior and “should” for recommended quality or operational behavior.

Term	**Meaning**
Community Skill Exchange Platform	The proposed web-based system described in this document.
User	A registered person using the platform.

Term	Meaning
Learner	A user seeking to acquire a skill or meet a learning goal.
Mentor	A user offering knowledge or guidance in one or more skills.
Skill Offered	A skill that a user is willing to teach or mentor.
Skill Needed	A skill that a user wants to learn.
Match	A potential mentor-learner pairing identified by the system.
Compatibility Score	A relative score used to rank the suitability of a match.
Mentorship Request	A request sent by a learner to a mentor.
Session	A scheduled interaction between a mentor and learner.
Learning Plan	A structured set of topics, activities, and milestones for a learning goal.
Session Summary	A concise record of topics discussed and agreed next steps.
AI Service	The intelligent service used for selected matching and content-generation functions.
FR	Functional Requirement.
NFR	Nonfunctional Requirement.
API	Application Programming Interface.
HTTPS	Hypertext Transfer Protocol Secure.
UML	Unified Modeling Language.
DFD	Data Flow Diagram.
ER	Entity Relationship model.

1.3 Intended Audience and Reading Suggestions

This document is intended for students implementing the project, Software Engineering faculty, project guides, UI/UX designers, database designers, software testers, and future maintainers. Readers who are evaluating the project should begin with Sections 1 and 2 for the system context, followed by Sections 3 and 4 for interfaces and functional behavior. Developers and testers should give particular attention to the requirement statements and Appendix B, where the structural and behavioral models are presented.

1.4 Product Scope

The product covers the complete basic workflow of community-based skill exchange. A registered user can create a profile, maintain skills offered and skills needed, define availability, search for suitable people, and review recommended matches. A learner can submit a mentorship request and, once accepted, work with the mentor through scheduled sessions. The system records session status, supports communication, provides feedback facilities, and keeps relevant notifications and activity records.

The AI-assisted part of the system is deliberately limited to functions that are useful within the scope of a short academic project: compatibility-based matching assistance, generation of an individual

learning plan, and generation of a session summary. The platform is not intended to replace human judgement in selecting a mentor or deciding whether a learning plan is suitable.

1.5 References

- Project specification: Community Skill Exchange and Mentorship Platform, SDAI project repository.
- Uploaded college SRS reference document used as the structural template for this specification.
- Object Management Group, Unified Modeling Language (UML) specification, for analysis and design notation.

2. Overall Description

2.1 Product Perspective

The proposed platform is a self-contained web application with a browser-based client, an application layer, a persistent database, and an AI service interface. The application layer is responsible for authentication, profile handling, skill management, matching, mentorship workflow, scheduling, communication, and administrative operations. AI-assisted operations are exposed through a separate service boundary so that the core system remains usable when the AI service is unavailable.

2.2 Product Functions

- user registration, login, authorization, and profile maintenance;
- availability and skill management for both teaching and learning interests;
- search and compatibility-based mentor/learner matching;
- match explanations and mentorship-request management;
- session scheduling, rescheduling, cancellation, and completion;
- messaging, notifications, and feedback;
- AI-assisted learning plans and session summaries;
- reporting, moderation, administration, and activity dashboards.

2.3 User Classes and Characteristics

User Class	Characteristics	Primary Functions
User	Registered participant. A user may take either or both roles depending on the activity.	Manage profile, skills, availability, searches, requests, sessions, messages, feedback and dashboard.
Learner	User who is currently looking for guidance or instruction in a skill.	Identify needed skills, find mentors, submit requests, schedule sessions, use learning plans and provide feedback.
Mentor	User who offers one or more skills and is willing to guide other participants.	Maintain offered skills, review matches, respond to requests, conduct sessions and receive feedback.
Administrator	Privileged user responsible for maintaining platform data and handling reports.	Manage users and skills, review reports, moderate activity and maintain system records.

2.4 Operating Environment

Category	Requirement
Client device	Laptop, desktop, tablet, or smartphone
Client RAM	4 GB minimum
Display	1366 × 768 or above recommended for desktop use
Network	Stable internet connection
Server processor	Intel Core i5 equivalent or above for a prototype deployment
Server RAM	8 GB or above
Server storage	20 GB or above
Operating systems	Windows, Linux, or macOS
Frontend	React / HTML / CSS / JavaScript / Streamlit
Backend	FastAPI / Flask / Node.js
Database	SQLite / PostgreSQL / MySQL
Browser	Chrome, Edge, or Firefox
Development tools	VS Code, Git, GitHub

2.5 Design and Implementation Constraints

- The application is dependent on browser and network availability.
- Authentication and administrator functions must be isolated from ordinary user access.
- The matching logic should continue to operate using stored profile data even when external AI services are unavailable.

- The quality of recommendations depends on the completeness and accuracy of user-entered skill and availability information.
- The final technology stack may be selected from the candidate technologies listed above according to project constraints.
- The prototype is intended for academic demonstration and is not assumed to have the capacity or operational controls of a large commercial platform.

2.6 User Documentation

The project should provide a concise user guide covering account creation, profile setup, skill entry, searching, reviewing matches, sending and responding to mentorship requests, scheduling a session, using AI-assisted features, and submitting feedback or reports. Administrator guidance should cover user management, report handling, and basic record maintenance.

2.7 Assumptions and Dependencies

- Users enter reasonably accurate profile, skill, and availability information.
- Mentors keep their offered skills and availability current.
- The database remains available while the system is in use.
- Where AI functions are enabled, the external or locally hosted AI service is reachable and accepts the required requests.
- The application is deployed in an environment with a supported browser and reliable network access.
- Any third-party AI or messaging service remains subject to its own availability and usage limits.

3. External Interface Requirements

3.1 User Interfaces

The interface should use a consistent navigation structure and separate ordinary user functions from administrative functions. The main screens are expected to include:

- registration and login;
- profile and availability management;
- skills offered and skills needed;
- search and match results;
- match explanation and mentorship request;
- request and session management;
- messaging, notifications, and feedback;
- learning plan and session summary;

- administrator dashboard and moderation screens.

3.2 Hardware Interfaces

No specialized hardware interface is required. Users access the system through standard computing or mobile devices with a supported web browser. Server operation requires standard compute, storage, and network resources.

3.3 Software Interfaces

Interface	Purpose	Candidate Technology
Web Browser	Presents the user interface and sends user requests.	Chrome / Edge / Firefox
Application Backend	Processes authentication, matching, mentorship workflow and other business logic.	FastAPI / Flask / Node.js
Database	Stores profile, skill, request, session and activity data.	SQLite / PostgreSQL / MySQL
AI Service	Supports matching assistance, learning-plan generation and session summaries.	AI API or local model service
Version Control	Stores source code and project history.	Git / GitHub

3.4 Communications Interfaces

Client devices shall communicate with the application server over HTTPS. The application server shall use the database connection mechanism supported by the selected database platform. Where an external AI provider is used, requests and responses should be exchanged over an authenticated API/HTTPS connection. Communication details specific to a final deployment environment remain subject to the chosen implementation.

4. System Features

The functional requirements are grouped below according to the major workflows of the proposed platform. Requirement identifiers FR1-FR29 are used consistently throughout the specification.

4.1 Account and Profile Management

4.1.1 Description and Priority

Provides the basic identity and profile information required for all subsequent platform functions. The profile stores personal information, the user role or roles, a short description, location or locality, and the preferred interaction mode.

Priority: High for the core academic prototype unless otherwise stated.

4.1.2 Stimulus/Response Sequences

Stimulus: A new visitor selects registration. Response: The system validates the submitted details and creates a user account.

Stimulus: A registered user enters valid credentials. Response: The system authenticates the user and opens the appropriate dashboard.

Stimulus: A user edits profile information. Response: The system validates and stores the revised profile data.

4.1.3 Functional Requirements

FR1: The system shall allow a new user to create an account using the required registration information.

FR2: The system shall authenticate registered users using valid login credentials.

FR3: The system shall restrict administrative functions to authenticated and authorized administrators.

FR4: The system shall allow a user to create and update profile information relevant to skill exchange and mentorship.

4.2 Availability and Skill Management

4.2.1 Description and Priority

Allows users to describe what they can teach, what they want to learn, and when and how they are available. This information forms the basis for meaningful profile comparison.

Priority: High for the core academic prototype unless otherwise stated.

4.2.2 Stimulus/Response Sequences

Stimulus: A user adds a skill they can teach. Response: The system stores the skill with proficiency or experience information.

Stimulus: A user adds a skill they want to learn. Response: The system records the learning goal and desired level.

Stimulus: A user updates available days or preferred mode. Response: The system updates the profile information used during matching.

4.2.3 Functional Requirements

FR5: The system shall allow users to maintain their available days and time ranges.

FR6: The system shall allow users to add, update, and remove skills they can teach.

FR7: The system shall allow users to add, update, and remove skills they want to learn.

FR8: The system shall allow users to record a preferred interaction mode such as online, offline, or either.

4.3 Search and Profile Matching

4.3.1 Description and Priority

Provides the main discovery mechanism of the platform. The system should identify users whose offered skills, learning goals, availability, and interaction preferences are sufficiently compatible with the current user.

Priority: High for the core academic prototype unless otherwise stated.

4.3.2 Stimulus/Response Sequences

Stimulus: A user searches for a skill or mentor. Response: The system returns profiles that satisfy the search criteria.

Stimulus: A user asks for recommended matches. Response: The system compares relevant profile attributes and returns a ranked set of compatible users.

Stimulus: A user opens a recommended profile. Response: The system displays the factors that contributed to the recommendation.

4.3.3 Functional Requirements

FR9: The system shall allow users to search for mentors, learners, and skills.

FR10: The system shall identify possible mentor-learner matches using skill compatibility, learning goals, availability, and interaction mode or location.

FR11: The system shall rank suitable matches using a compatibility score or equivalent ranking method.

FR12: The system shall provide a concise explanation of the main factors contributing to a recommended match.

4.4 Mentorship Request and Session Management

4.4.1 Description and Priority

Controls the transition from a potential match to an actual mentorship relationship and provides the scheduling workflow for individual sessions.

Priority: High for the core academic prototype unless otherwise stated.

4.4.2 Stimulus/Response Sequences

Stimulus: A learner selects a mentor and submits a request. Response: The system creates a pending mentorship request.

Stimulus: A mentor accepts or rejects a request. Response: The system updates the request status and notifies the learner.

Stimulus: Both participants select a mutually suitable time. Response: The system creates a session after checking for conflicts.

Stimulus: A scheduled meeting is completed. Response: The session status is updated and the system enables post-session actions.

4.4.3 Functional Requirements

FR13: The system shall allow a learner to send a mentorship request to a selected mentor.

FR14: The system shall allow the mentor associated with a request to accept or reject it.

FR15: The system shall maintain request states including Pending, Accepted, Rejected, Cancelled, and Completed where applicable.

FR16: The system shall allow accepted mentor-learner pairs to schedule a session.

FR17: The system shall check the relevant availability information before confirming a session.

FR18: The system shall support session states such as Scheduled, Rescheduled, Cancelled, and Completed.

4.5 Communication and Notifications

4.5.1 Description and Priority

Supports communication needed to coordinate mentorship activity and keeps users informed about changes to requests, sessions, and other relevant actions.

Priority: High for the core academic prototype unless otherwise stated.

4.5.2 Stimulus/Response Sequences

Stimulus: A participant sends a message. Response: The system stores the message and makes it available to the intended recipient.

Stimulus: A request or session changes state. Response: The system creates a notification for the affected participant.

4.5.3 Functional Requirements

FR19: The system shall allow a mentor and learner to exchange messages within the supported mentorship context.

FR20: The system shall notify users about important request, session, and platform events.

4.6 AI-Assisted Learning Plan and Session Summary

4.6.1 Description and Priority

Provides limited AI assistance at points where structured text generation can reduce repetitive work. The AI output should be grounded in the information supplied by the users and should remain editable or reviewable before it is treated as part of the user record.

Priority: High for the core academic prototype unless otherwise stated.

4.6.2 Stimulus/Response Sequences

Stimulus: A learner requests a plan for a learning goal. Response: The AI service receives the relevant goal and skill context and returns a structured sequence of topics and activities.

Stimulus: A completed session is selected for summarization. Response: The AI service returns a concise summary with key points and next steps.

Stimulus: The AI service is unavailable or returns an unusable result. Response: The application reports the failure and preserves the underlying session or profile data.

4.6.3 Functional Requirements

FR21: The system shall use relevant learner profile and goal information when requesting an AI-generated learning plan.

FR22: The system shall present the generated learning plan in an understandable structure that can be reviewed by the user.

FR23: The system shall generate a session summary containing, where supported, topics discussed, key points, and suggested next steps.

FR24: The system shall handle AI-service failure without preventing access to the underlying profile, request, or session records.

FR25: The system shall identify AI-generated content as such and shall not represent generated guidance as a guaranteed outcome.

4.7 Feedback, Reporting and Administration

4.7.1 Description and Priority

Provides basic mechanisms for quality control and moderation. Participants can record feedback after completed sessions, while administrators can review reports and maintain system data.

Priority: High for the core academic prototype unless otherwise stated.

4.7.2 Stimulus/Response Sequences

Stimulus: A completed session participant submits a rating and comment. Response: The system associates the feedback with the relevant session and participant.

Stimulus: A user submits a report. Response: The system records the report for administrative review.

Stimulus: An administrator opens the moderation area. Response: The system presents authorized management functions and relevant records.

4.7.3 Functional Requirements

FR26: The system shall allow users to submit feedback after a completed mentorship session.

FR27: The system shall allow users to submit a report concerning inappropriate users, content, or platform activity.

FR28: The administrator shall be able to manage users, skills, mentorship requests, sessions, reports, and related platform records.

Administrative and dashboard behavior that complements FR26 is specified through the requirements and models in the remaining sections of this SRS. The system shall additionally provide reporting and administrator controls needed to moderate basic platform activity.

4.8 Dashboard and Activity Tracking

4.8.1 Description and Priority

Provides a consolidated view of information that a user is likely to need after logging in. The dashboard should reduce the need to navigate through separate pages to understand current requests, upcoming sessions, recent matches, and AI-assisted outputs.

Priority: High for the prototype because the dashboard is the main entry point after authentication.

4.8.2 Stimulus/Response Sequences

Stimulus: A user opens the dashboard. Response: The system retrieves the latest profile status, match suggestions, open requests, scheduled sessions, notifications, and available learning-plan information.

4.8.3 Functional Requirements

FR29: The system shall provide a personalized dashboard containing relevant matches, mentorship requests, upcoming sessions, notifications, and recent activity.

The administrator shall be able to view basic operational records and manage authorized platform data.

5. Other Nonfunctional Requirements

5.1 Performance Requirements

- The system should respond to normal navigation, profile, and database operations within a few seconds under the expected project load.
- Search and matching operations should return a usable result without requiring repeated requests from the client.
- AI-assisted operations should display a progress or status indication when processing takes longer than normal page operations.
- The application should support simultaneous access by multiple users within the limits of the selected prototype deployment.

5.2 Safety Requirements

No specialized physical safety requirements are identified because the platform is a conventional web application. The system should, however, avoid presenting AI-generated learning guidance as authoritative or guaranteed. Users should remain able to review and disregard generated suggestions.

5.3 Security Requirements

- Passwords shall not be stored in plain text.
- Protected user information shall only be available to authenticated users with appropriate permissions.
- Administrative operations shall require administrator authorization.
- The application shall use HTTPS for client-server communication in deployment.
- Session and authentication data shall be protected against unauthorized reuse.
- User reports and moderation records shall not be exposed to ordinary users unless explicitly intended by the application design.

5.4 Software Quality Attributes

Quality Attribute	**Requirement**
Reliability	The system should preserve user, request, session, and feedback records during normal operation.
Availability	Core platform functions should remain available whenever the application and its supporting services are operational.
Usability	Common tasks should be understandable without requiring technical knowledge of the underlying implementation.
Scalability	The design should allow growth in users, skills, requests, and sessions without redesigning the entire system.
Maintainability	The application should use modular components and clear interfaces so individual services can be changed independently.
Portability	The web application should run on commonly used operating systems and modern browsers.
Explainability	The main factors contributing to a recommendation should be visible to the user.
Testability	Each major workflow should be independently testable through unit, API, and integration tests.

5.5 Business Rules

- A registered user may participate as a mentor, learner, or both.
- A user shall not create a mentorship request addressed to the same account.
- A mentor may respond only to requests associated with that mentor account.
- A session should not be confirmed when the selected time conflicts with the availability recorded by the participants.
- Feedback is intended for completed sessions.
- Only administrators may perform moderation and other privileged management actions.

- AI-generated results are advisory and should be reviewable by the user.
- When an AI service fails, the platform shall retain the underlying non-AI workflow and data.
- Private profile and message information shall be visible only to users authorized to access it.

6. Other Requirements

The system requires persistent storage for user accounts, profile information, skills offered, skills needed, mentorship requests, sessions, messages, feedback, notifications, reports, and AI-generated learning-plan or session-summary records. The conceptual model is designed so that these records can be implemented in a relational database. SQLite may be used for a local academic prototype, while PostgreSQL or MySQL may be selected for a hosted implementation.

The final implementation should also define validation rules for email addresses, passwords, skill entries, scheduling times, request states, and feedback. The exact data-retention policy, external AI provider, deployment configuration, and any additional privacy or compliance requirements are project decisions and are therefore captured in Appendix C.

Appendix A: Glossary

Term	Definition
Mentor	A registered user who offers knowledge, guidance, or practical help in one or more skills.
Learner	A registered user seeking to improve a skill or achieve a stated learning goal.
Skill Offered	A skill recorded by a user as an area in which they are willing to teach or mentor.
Skill Needed	A skill recorded by a user as an area in which they want to learn.
Match	A potential mentor-learner pairing identified from compatible profile information.
Compatibility Score	A relative score representing how closely two profiles fit the matching criteria.
Mentorship Request	A request through which a learner asks a mentor to provide guidance.
Session	A scheduled interaction between a mentor and learner.
Learning Plan	A structured sequence of topics, tasks, and milestones associated with a learner goal.
Session Summary	A concise record of the main discussion points and next steps from a mentorship session.
AI Service	A service used to produce selected AI-assisted results such as matching support or structured text.
Dashboard	A consolidated screen showing the current user's most relevant information and actions.
API	Application Programming Interface used for communication between software components.
HTTPS	Secure communication protocol used between clients and web servers.
DFD	Data Flow Diagram representing movement of information through the system.
ER Model	Entity Relationship model describing data entities and their relationships.
UML	Unified Modeling Language used for software analysis and design diagrams.

Appendix B: Analysis Models

This appendix presents the principal analysis and design models for the proposed platform. The models are intended to remain consistent with the requirements described in Sections 3 and 4.

B.1 Use Case Model

The primary actor is the User, who can perform mentor and learner activities. The Administrator has privileged moderation and management functions. The AI Service acts as a supporting system actor for matching explanation, learning-plan generation, and session summarization.

Important use cases include registration/login, profile management, skill management, matching, mentorship requests, session scheduling, communication, AI-assisted content generation, feedback, notifications, and administration.

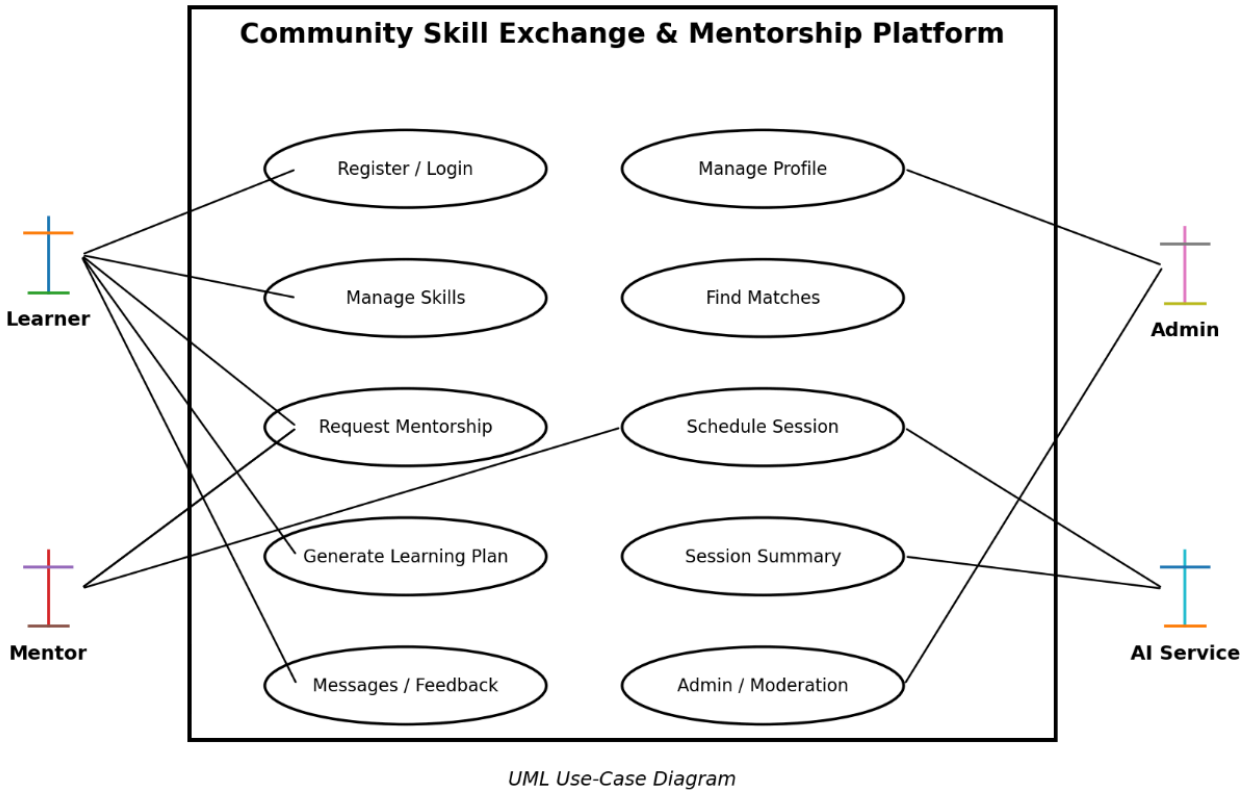


Figure B.1: Use Case Diagram

B.2 Class Model

The class model identifies the principal classes required by the application logic. User and Skill form the base profile structure, while MentorshipRequest and Session represent the operational workflow. AIService is separated from the domain classes so that AI-provider changes do not require changes throughout the core application.

Class	**Attributes**	**Methods**
User	userId, name, email, passwordHash, bio, role, location, preferredMode, availability	register(), login(), updateProfile(), setAvailability(), viewDashboard()
Skill	skillId, skillName, category, description	searchSkill(), updateSkill()
SkillProfile	offeredSkills, neededSkills, proficiency, targetLevel, learningGoals	addOfferedSkill(), addNeededSkill(), calculateCompatibility()
MentorshipRequest	requestId, learnerId, mentorId, skillId, message, status, createdAt	sendRequest(), acceptRequest(), rejectRequest(), updateStatus()
Session	sessionId, requestId, date, startTime, endTime, mode, status	schedule(), reschedule(), cancel(), complete()
Message	messageId, senderId, receiverId, messageText, sentAt	send(), receive()
Feedback	feedbackId, sessionId, giverId, rating, comment, createdAt	submitFeedback()
AIService	service interface only	matchProfiles(), explainMatch(), generateLearningPlan(), summarizeSession()

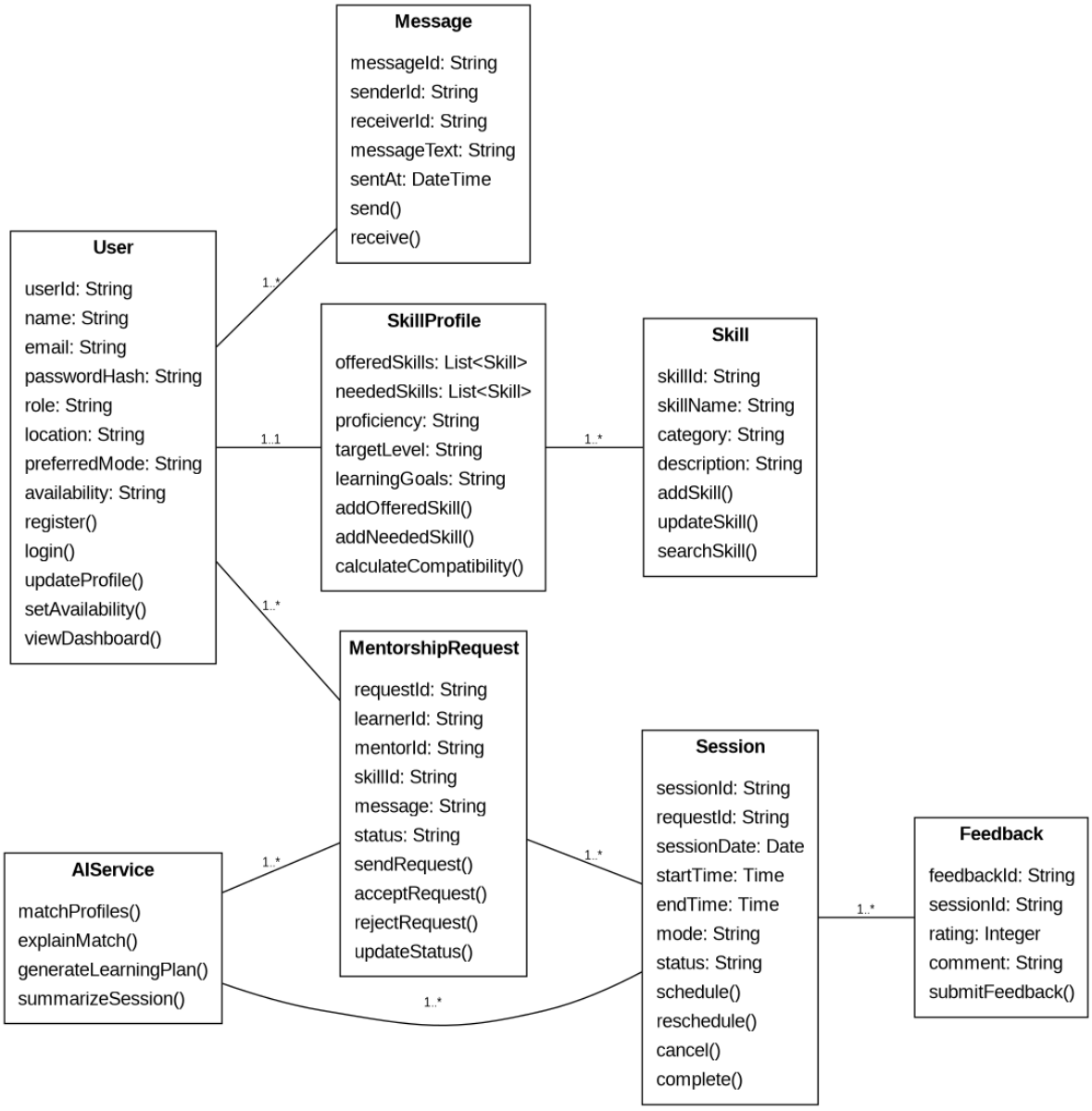


Figure B.2: Class Diagram

B.3 Object Model

The object model shows a representative runtime configuration. The values are illustrative and demonstrate the relationship between a learner, mentor, skill records, a mentorship request, a session, and a learning plan.

Object	Example Values	Connected To
user1 : User	U101; Learner; Delhi; Online	need1, request1, plan1
user2 : User	U102; Mentor; Delhi; Online	offer1, request1
skill1 : Skill	SK01; Python	offer1, need1
offer1 : SkillOffered	U102 / SK01; Advanced	user2, skill1
need1 : SkillNeeded	U101 / SK01; Intermediate	user1, skill1

Object	Example Values	Connected To
request1 : MentorshipRequest	MR101; Accepted	user1, user2, session1
session1 : Session	S101; Scheduled	request1
plan1 : LearningPlan	LP101; Python for Data Analysis	user1

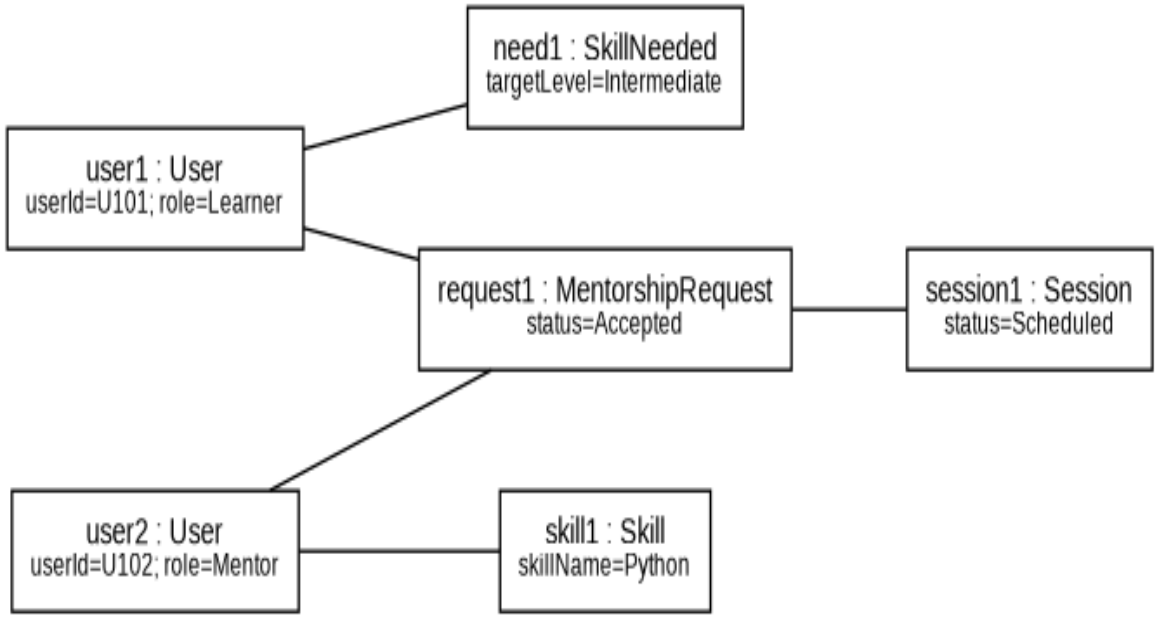


Figure B.3: Object Diagram

B.4 Package Model

Package	Contents
User Management	Authentication, User, Profile, Availability
Skill Management	Skill, Skill Offered, Skill Needed
Matching	Compatibility logic, Ranking, Match Explanation
Mentorship	Mentorship Request, Session, Scheduling
Communication	Messages, Notifications
AI Services	Profile Matching, Learning Plan, Session Summary
Feedback & Reports	Ratings, Feedback, Reports, Moderation
Administration	User management, Skill management, Reports, Audit
Database	Persistent records used by the application modules

Package dependencies are directed from user-facing management modules toward skill and matching services, from matching and mentorship modules toward AI services where required, and from operational modules toward the database layer.

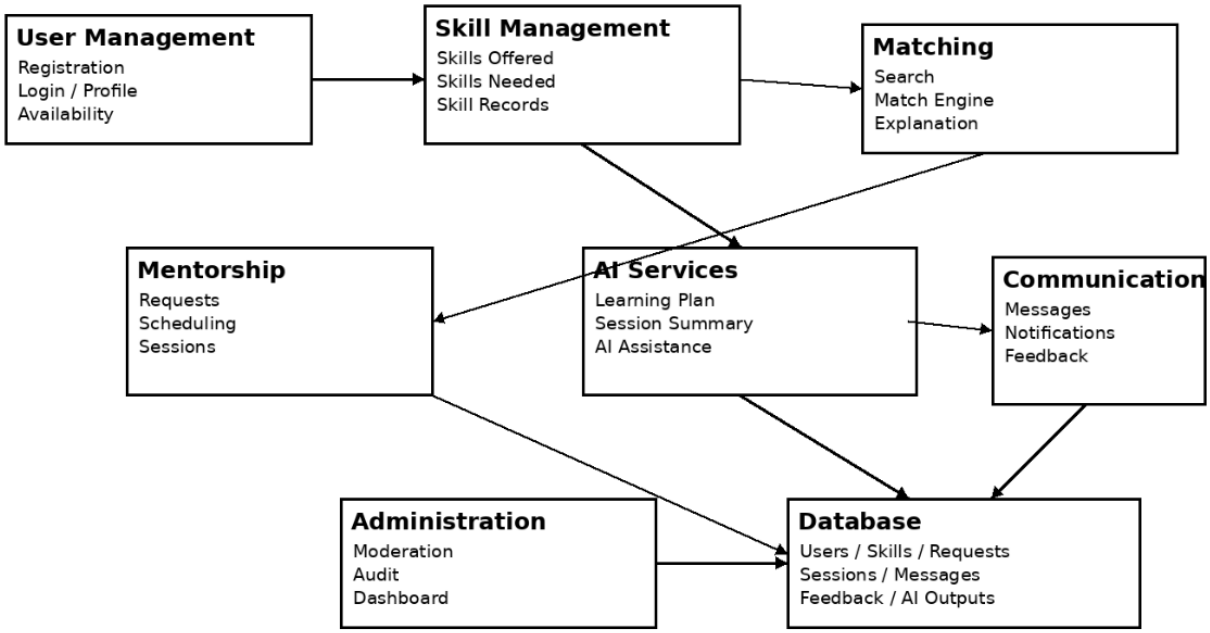


Figure B.4: Package Diagram

Figure B.4: Package Diagram

B.5 Component Model

The component model separates the user interface from authentication, profile, skill, matching, mentorship, scheduling, communication, AI, feedback, administration, and persistence services. This

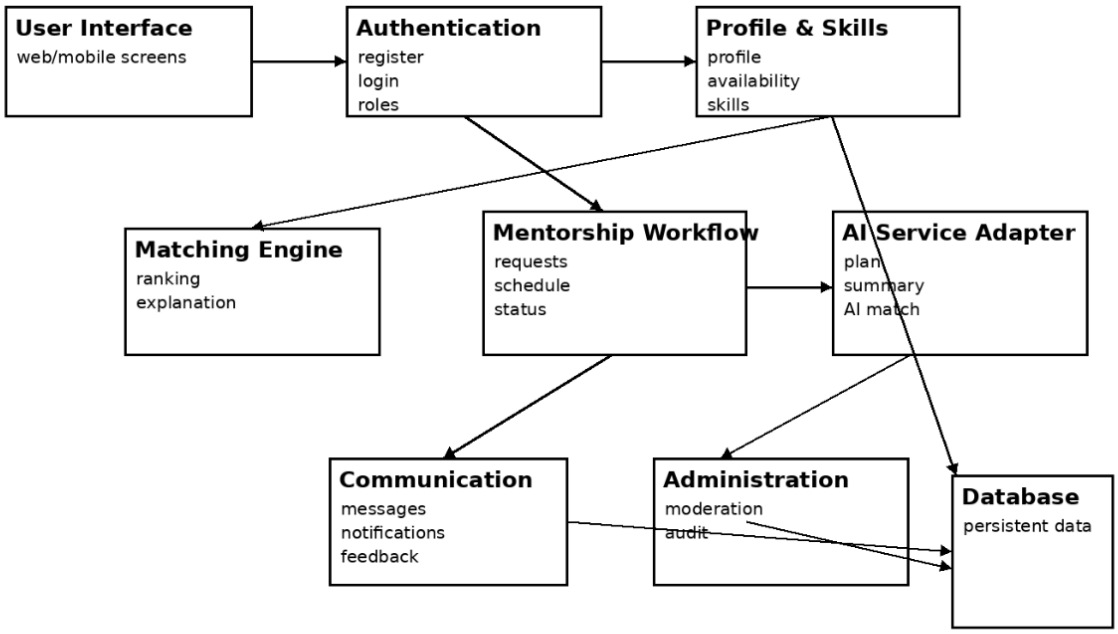


Figure B.5: Component Diagram

structure supports independent testing and makes future changes to the AI provider or database layer easier.

Figure B.5: Component Diagram

B.6 Deployment Model

Node	Role
User Device	Laptop, desktop, tablet, or smartphone running a supported web browser.
Application Server	Hosts the web application and server-side business logic, including authentication, profile management, matching, mentorship workflow, and API endpoints.
Database Server	Stores persistent user, skill, request, session, message, feedback, notification, and report records.
AI Service	Provides selected AI-assisted functions through an API or equivalent service boundary.

Deployment connections: User Device --HTTPS--> Application Server; Application Server --Database Connection--> Database Server; Application Server --API/HTTPS--> AI Service.

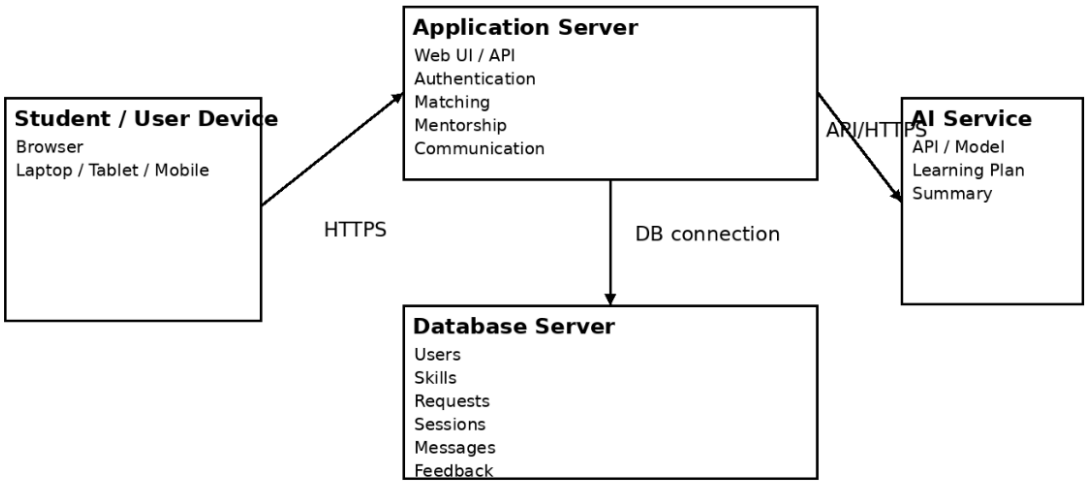


Figure B.6: Deployment Diagram

Figure B.6: Deployment Diagram